

Report/Documentation

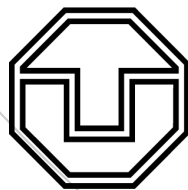
TEAM PROJECT: NAPARI-DASK INTEGRATION

Artem Tomilo

Nafisa Anjum

Himanshu Manoj Kaloni

September 4, 2023



**TECHNISCHE
UNIVERSITÄT
DRESDEN**

This work is licensed under a Creative Commons “Attribution-NonCommercial-NoDerivatives 4.0 International” license.



Contents

1	Introduction	1
2	Bio-Image Analysis	2
3	A major problem of bio-image analysis we are addressing in our project:	4
4	Programming language and libraries	5
	References	7

1

Introduction

In today's biology, we encounter intricate challenges like understanding protein folding, interpreting the factors influencing cell growth, and unraveling the complex mechanisms behind the actions of drugs. Data about a computational or natural experiment in biology is very often stored in the form of high-resolution microscopic images. These images can range from cellular and sub-cellular structures captured through microscopy to medical imaging data obtained from MRI, CT scans, and more. The analysis of such images is crucial for advancing our understanding of biological systems, disease mechanisms, and therapeutic interventions. Manual inspections for bio-image analysis are frequently performed by biologists. A biologist, for example, might track an organelle by watching all video frames with his or her eyes and analyzing its movement in a cell, which takes a lot of effort and concentration. It takes far more effort if there are multiple target organelles. As a result, analyzing many bio-images and producing reliable analysis results is difficult. As a result, biologists face challenges in image processing due to the size of the image data which impacts their ability to extract meaningful information from those images.

Microscopy-based scientific findings can be supported by quantitative image analysis. But imaging results are 3D volumetric images and 4D volumetric videos which are large-scale image data. Due to this reason, biologists face challenges in image processing which impacts their ability to extract meaningful information from those images. Addressing those problems requires interdisciplinary collaborations between biologists and computer scientists.

At its core, Napari is built using Python, a popular and widely-used programming language, which grants it a significant advantage in terms of accessibility and extensibility. Its architecture is designed to seamlessly integrate with numerous powerful Python libraries and packages, transforming it into an invaluable asset for scientists, particularly those engaged in fields such as biology, microscopy, and image analysis.

Bio-Image Analysis

- What is Bio-image analysis?

The process of extracting numerical data from scientific images is known as bio-image analysis. There are numerous software packages, methods, and algorithms available that enable various types of analysis. [3]

- How do computers see an image?

The fundamental building blocks of an image are its pixels, a term derived from "picture element." In terms of computing, each pixel is essentially a numerical value. While we typically perceive images as collections of colored squares when displayed, this visual representation is primarily for our convenience. It provides us with a quick way to grasp the image's content, including approximate pixel values and their spatial relationships. However, for the purposes of processing and analysis, we must transition beyond the visual display and engage with the underlying data, which consists of numerical values. In summary, Computers perceive images as collections of numerical data arranged in a grid of pixels.

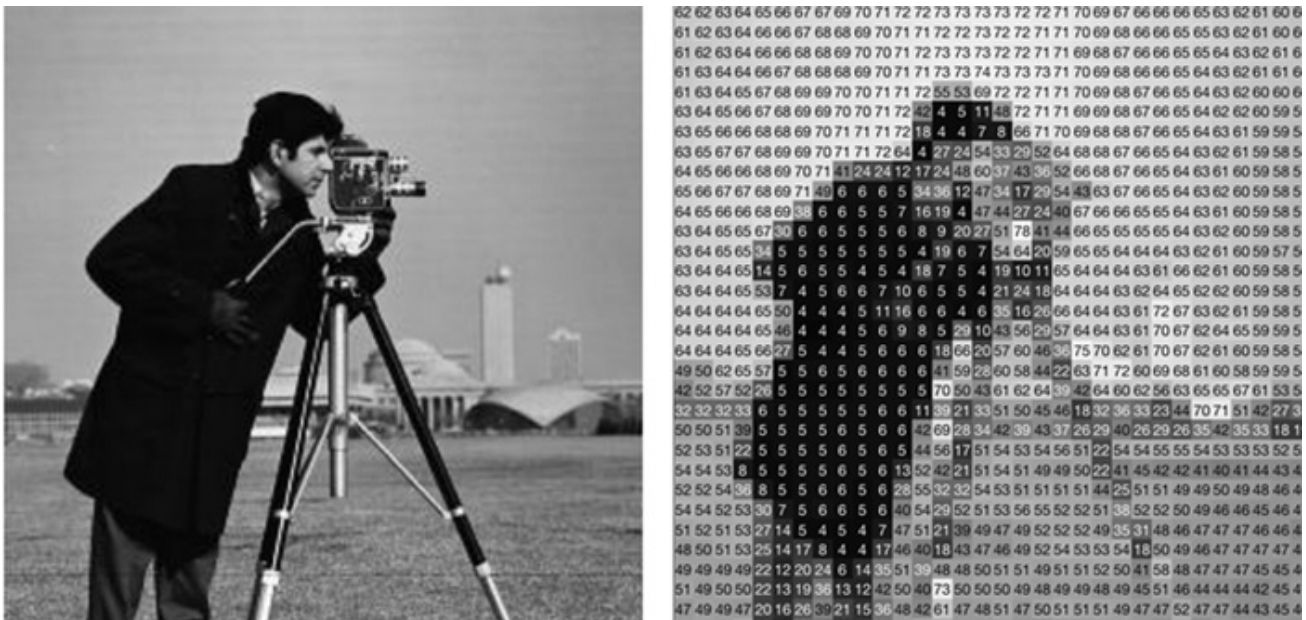


Figure 1: This picture is a comparison how human sees an image vs. the computer
 Source: : (<https://www.expressanalytics.com/blog/10-things-about-computer-vision-and-data-analytics-you-may-not-know/>)

- **Pipeline of Bio-image analysis**

A basic pipeline for bio-image analysis involves a series of steps that process and extract information from biological images. While the specific details can vary depending on the nature of the images and the research objectives, here's a simplified outline of a typical bio-image analysis pipeline:

- Image Acquisition
- Image Preprocessing
- Image Segmentation
- Feature Extraction
- Object Classification and Filtering
- Data Analysis
- Visualization and Interpretation

Bio-image analysis pipelines can be highly customized to suit specific research needs, and they often involve a combination of software tools, scripting, and domain-specific knowledge. Additionally, the choice of algorithms and techniques may vary widely depending on the type of biological samples, imaging modalities, and research goals.

3

A major problem of bio-image analysis we are addressing in our project:

Dealing with Data Volume: Modern imaging techniques generate large-scale image data making processing, analyzing, and storing computationally challenging. Handling large data sets efficiently requires high computational costs.

- **How we are addressing the problem**

To address this issue and efficiently process, analyze, and store large-scale image data we are using Parallel and distributed computing.

- **Parallel and distributed computing**

Parallel and distributed computing are computing paradigms that involve the simultaneous execution of tasks across multiple processors or computers to improve performance, solve complex problems, and handle large-scale data efficiently. Distributed computing extends parallel computing to multiple computers or nodes connected through a network. Tasks are distributed across these nodes to work in tandem, addressing problems that exceed the capabilities of a single machine.

In the realm of parallel computing, the primary objective is to harness the combined processing power of multiple CPU cores or processors within a single machine. By splitting tasks into smaller, manageable chunks and executing them concurrently, parallel computing significantly enhances computational speed and efficiency.

Distributed computing, on the other hand, takes the concept of parallelism to a grander scale by seamlessly integrating multiple computers or nodes connected over a network. Each node in a distributed system contributes its processing capabilities and memory resources to work collaboratively on a common task or problem. This distributed architecture opens up the door to solving problems that are beyond the capacity of a single machine. Moreover, it offers the advantages of fault tolerance and scalability. Distributed computing finds extensive use in cloud computing, web services, and scenarios where data must be processed across geographically dispersed locations.

Programming language and libraries

In this project, we proposed a solution to address this problem using programming libraries namely Dask (parallel and distributed processing) and napari (Interactive Image processing). The software project will be written in Python, which is an easy-to-learn scripting language for solving a wide range of development problems. By integrating Dask with Napari, we can distribute image processing tasks across multiple computers, thereby accelerating the analysis and manipulation of large-scale image data sets.

Python has a wide array of libraries and packages specifically designed for image analysis, making it a powerful platform for this purpose. [4]

Some of the most notable libraries include:

- OpenCV :

OpenCV is one of the most popular and comprehensive libraries for computer vision and image processing. It provides a vast collection of functions for tasks like image manipulation, feature detection, object recognition, and more.

- Pillow (PIL Fork) :

Pillow is a user-friendly library for basic image processing tasks, such as opening, editing, and saving various image formats.

- Scikit-Image:

Scikit-Image is part of the Scikit-Learn ecosystem and offers a wide range of tools for image segmentation, feature extraction, and image analysis.

- Mahotas:

Mahotas is a library for computer vision tasks, particularly suited for tasks like image thresholding, edge detection, and texture analysis.

Dask is a versatile and user-friendly library that offers scalable, parallel computing capabilities, making it an excellent choice for data scientists, researchers, and engineers working on tasks involving large data sets, distributed computing, and parallelism in Python. Its seamless integration with existing Python libraries and its ability to handle big data efficiently make it a valuable addition to the data processing and analysis toolkit. [1]

Napari is an exceptionally versatile and user-friendly open-source software that serves as an interactive and multi-dimensional image viewer and analysis tool. Developed with a strong fo-

cus on meeting the specific needs of the scientific community, Napari has quickly become an indispensable resource for researchers across various disciplines. At its core, Napari is built using Python, a popular and widely-used programming language, which grants it a significant advantage in terms of accessibility and extensibility. Its architecture is designed to seamlessly integrate with numerous powerful Python libraries and packages, transforming it into an invaluable asset for scientists, particularly those engaged in fields such as biology, microscopy, and image analysis.[2]

References

- [1] DASK. In: (). URL: <https://www.dask.org/>.
- [2] NAPARI. In: (). URL: <https://napari.org/stable/>.
- [3] Biopolis Dresden Imaging PLATFORM. In: (). URL: <https://www.biodip.de/resources/bioimage-analysis/index.html#:~:text=Bioimage%20analysis%20is%20the%20process,and%20information%20related%20to%20analysis..>
- [4] PYTHON. In: (). URL: <https://www.python.org/>.